Flint Q8 Benchmark: technical report

Jaime Lawson | Founder & Research Lead, Phoenix Applied Technologies

Public benchmark contact: jmlawson@outlook.com

Release package v5 | November 5, 2026 | Public release approved; publication pending

Original research documentation: Copyright 2026 Jaime Lawson, Phoenix Applied Technologies. Licensed under CC BY 4.0; third-party material excluded. See LICENSING.md.

1. Scope and methodology

This report condenses saved Colibri experiments on Phoenix. It separates throughput, response latency and correctness. The publication revision runs no model inference and changes no production configuration.

System: Debian 13, Ryzen 7 2700X, approximately 78 GiB RAM, RTX 4060 8 GB and GTX 1080 Ti 11 GB. Earlier dual-GPU stages used a **GTX 1070 8 GB**, explicitly identified below. The later selected setup uses eight OpenMP inference workers; helpers add other process threads.

Model/runtime: `unsloth/flint-GGUF`, Q8_0, 42,301,076,928 bytes (42.30 GB; 39.40 GiB); Colibri v2.1.0, `qwen36`. Native GGUF weights were read in place without requantization. The source commit, model revision and hashes are retained in E8. The CUDA binary contains Pascal `sm_61` and Ada `sm_89` targets.

The main explanation prompt asks how insulation slows heat transfer in about 250 words, with no tools. It used 37 prompt tokens. The distinct async-cache repair used 491 prompt tokens. Exact prompts, answers and grading criteria remain in the supporting evidence. Historical generation used the visible Colibri composer; fresh conversations were used except for labeled quality follow-ups.

Measure	Meaning	Boundary
Completed decode rate	Completed tokens / engine decode-loop seconds	Includes reasoning; excludes prompt phase
Sampled decode rate	Last active cumulative health average	May omit the generation tail
End-to-end interval	Send to last active / first idle sample	Includes prompt processing and UI dispatch
Prompt phase	Health `read_s`	Not direct browser first-token timing

Later sampling was approximately every 0.75 seconds plus collection overhead; early sampling was approximately every two seconds. Intervals are observed sampling bounds, not confidence intervals. Full-response ceilings were 6,144 tokens with 8,192-token context; short screens used 256 or 768 tokens. The UI Reasoning checkbox did not reliably suppress reasoning, which was present in exports.

Later controlled arms started from separate copies of frozen expert history. Filesystem/page-cache coldness was not controlled. Saved history may favor the repeated prompt. Early reported temperature-zero settings have a UI-state caveat; the later cache and GPU-role pairs verified the visible 0.0 label. Exact request payloads were not recovered for the early screens.

Credit: Colibri developers, including Vincenzo Fornaro; Recursive (Flint); Unsloth (GGUF). Codex/Sapphire assisted preparation and verification, not independent replication. See CREDITS.md and LICENSING.md.

2. Throughput, latency and progression

Complete responses

Configuration / workload	tok/s	Method	Send-to-completion
4060 only: insulation	16.171	Sampled	213.85-215.85 s
4060 + 1070: Vulkan insulation	17.691	Sampled	76.94-78.98 s
4060 + 1070: tuned CUDA insulation	25.279	Sampled	87.61-88.40 s
4060 + Ti: CUDA control A	30.464	Completed	61.15-61.94 s
4060 + Ti: CUDA control B	30.567	Completed	60.14-60.94 s
4060 + Ti: coding repair	20.418	Completed	242.71-243.50 s

Control A produced 1,819 tokens in 59.709215 decode seconds; control B produced the same count in 59.508387 seconds. Exported final text and reasoning match. These two same-system, same-prompt observations support repeatability on that question, not independent replication or general throughput. Their prompt phases were 1.065 / 1.077 seconds. [E1, E2]

The coding run produced 4,569 tokens in 223.770323 decode seconds, plus a 19.278-second prompt phase: about **4:03 overall**. Thus the practical comparison is **30.6 tok/s on the short explanatory input** versus **20.4 tok/s on the coding repair**, both with full responses and reasoning included. [E3]

Short CUDA/Vulkan screen: 4060 + 1080 Ti

Each arm ran once at a 768-token ceiling and stopped during reasoning. All rates below are sampled averages, not completed-answer rates. [E4]

Configuration	GPU expert slots: 4060 + Ti	tok/s
Fixed-budget CUDA	1,253 + 1,762	27.830
Automatic-budget CUDA	1,377 + 2,532	**31.194, preliminary**
Vulkan baseline	960 + 1,700	20.574
Expanded Vulkan cache	960 + 2,688	23.378

Automatic CUDA measured 12.1% above fixed CUDA in this screen. Its 31.2 rounded headline remains preliminary: no same-protocol independent repeat and unresolved effective-temperature verification. Later full-response controls do not validate this separate short-screen result.

Across the earlier stages, hardware, backend, cache state and output length changed. The large reduction in full-answer latency partly reflects shorter reasoning. No hardware-only speedup or universal CUDA/Vulkan ranking is claimed. [E5]

3. Optimization, memory and answer quality

What tuning established

The 4060 retained the dense/attention chain; both GPUs held expert caches, with substantial CPU/RAM involvement. This is component/cache placement, not a whole-layer workload percentage. Automatic allowances vary with available VRAM and desktop use.

On the earlier 4060 + 1070 sweep, four workers were slower; six versus eight was close to observed variation. Eight were retained, without claiming a proven optimum. Two CUDA screens at the 5.5 allowance averaged 26.889 tok/s, versus 25.857 at 5.0, about 4.0% higher. A forced 80/20 expert-cache split reduced total resident capacity and slowed both backends; it did not isolate scheduling balance. [E5]

Later full-response comparisons used visibly verified temperature 0.0 and copied frozen heat seeds. Prompt-aware replanning on measured 26.121 versus 30.464 tok/s off (**14.3% lower**). Ti-primary dense computation measured 27.207 versus 30.567 on the 4060 (**11.0% lower**). Output lengths and automatic cache counts differed slightly. The Ti-primary answer finished sooner because it generated fewer tokens, despite slower decode. [E1, E2]

Expert history loads at startup and changes on graceful exit. Frozen numeric seeds are retained separately; each repetition must reset its working copy. Startup warmstart times are not cold-disk benchmarks. No isolated history-on/history-off ablation was performed.

The later 4060-primary control reached **40.14 GiB engine RSS**, with sampled total device VRAM peaks of **7,004 MiB / 9,531 MiB**. VRAM includes desktop allocations; RSS and system available RAM have different scopes. These are sampled peaks, not guaranteed maxima or model-only allocations. No energy-efficiency claim is supported. [E2]

Successes and failures

Saved coding implementation	Passed	Failed
Original buggy async cache	5	2
Completed single-GPU Flint patch	7	0
Later dual-GPU Flint patch	7	0

The fix guards stale-task cleanup with the generation/epoch check. Only patch formatting was normalized. The original first attempt truncated at 2,048 tokens before a final answer; the prompt was then resubmitted with a larger budget. Both successful runs concern the **same problem**. Seven test passes are not seven independent model tasks. The published commands were replayed from a clean repository copy; outputs and exit statuses are retained. [E6]

Physics failure: saved insulation answers misleadingly describe thermal pads/interfaces as barriers that improve GPU cooling by slowing heat toward the cooler. The earlier Vulkan answer also reverses contact-resistance reasoning. Later controls include a caution against blocking heat but retain confusing examples; no accuracy improvement is inferred. The five-case targeted set records four practical passes and one failure, including dependent follow-ups; this is not an overall accuracy percentage. [E7]

4. Reproduce, verify and interpret

Verified offline commands

From the **repository root**, with Python 3.8+ on Linux (verified with 3.13.5) and no third-party Python packages:

```
python3 tools/verify_coding.py
python3 supporting-evidence/scripts/rebuild_tables.py --check
```

The coding runner copies the unchanged fixtures into a disposable directory, applies process limits and timeouts, and reports every result. Overall success requires the buggy baseline to fail exactly two tests and both saved patches to pass all seven. The second command checks saved evidence and table arithmetic; neither invokes a model.

Direct commands, from `supporting-evidence/tests/async-cache/`:

```
PYTHONDONTWRITEBYTECODE=1 python3 test_cache.py buggy_cache
PYTHONDONTWRITEBYTECODE=1 python3 test_cache.py flint_fixed_cache
PYTHONDONTWRITEBYTECODE=1 python3 test_cache.py q8_fixed_cache
```

Expected exit statuses are **1, 0, 0**, respectively. Baseline failure is intentional and visible. Working-directory ambiguity is removed by the root runner; the historical tests and generated fixes are unchanged. See verification outputs.

Inference reproduction and limits

Use an isolated pinned Colibri checkout and model, separate state per arm, and the saved CUDA/Vulkan settings. Reset the same frozen seed, verify GPU ordering and effective UI/request temperature, record actual expert slots, then retain Send times, raw samples, completed profiles and exported responses. Keep capped screens separate from full answers. The full historical commands, model SHA-256, source revision and configuration records remain in the reproduction appendix.

Independent repetition of 31.2 remains future work. Retain all repeats and use fresh unseen tasks for broader claims. Historical driver/kernel/compiler patch versions and the complete original dual-CUDA compile command are missing; exact binary regeneration is incomplete. Page-cache state, background load and changing output length limit cross-stage comparisons. Two loaded transition runs are excluded but preserved. The initial coding Send timestamp was not found as a separate event. Evidence remains at the prior audit snapshot; later Q6 work is outside scope. No long-context, thermal-soak, broad accuracy or Q6 comparison claim is made.

Evidence key

Paths below are relative to `supporting-evidence/`; original records remain separate and intact.

- **E1:** `evidence/prefill-cache-1080ti/` - completed cache profiles, samples and comparisons.
- **E2:** `evidence/gpu-role-1080ti/` - completed controls, memory and matching-output record.
- **E3:** `evidence/dual-gpu-coding/` - completed coding profile, latency and grading.
- **E4:** `evidence/tuned-1080ti/` - capped CUDA/Vulkan measurements and exclusions.
- **E5:** Under `evidence/`: `early-backends-1070/`, `thread-cache-1070/`, `split-8020-1070/` - historical stages.
- **E6:** `tests/async-cache/`; `evidence/single-gpu-coding/` - fixtures, patches and initial truncation.
- **E7:** `evidence/quality-cases/`; `docs/QUALITY.md` - grades, failures and physics reference.
- **E8:** `evidence/model-validation/status.json`; `data/publication-environment.json` - identity and hashes.

Jaime has approved public release. Personal paths and network identifiers are redacted; the full prior sanitized evidence tree is unchanged. This concise version adds no inference results.